# CSE 3206 Reflection Essay

## EduPredict – AI Student Performance Predictor

**Submitted by**
**Mashfiq Ahmed Naushad**
Roll: 2103056

**Submitted to**
**Farjana Parvin**
Assistant Professor
Dept. of Computer Science & Engineering, RUET

February 9, 2026

**Abstract**

This reflection analyzes my self-directed learning in the EduPredict project, a full-stack system that predicts student risk and provides explainable results to teachers, students, and administrators. The central technical decision was to integrate SHAP explainability with a LightGBM classifier inside a simple ensemble, and to expose explanations through an API. I justify this decision by linking model behavior to stakeholder requirements, code structure, latency constraints, and data governance. I also compare the choice against alternatives with respect to scalability, interpretability, and operational cost. Finally, I evaluate my learning process and present a concrete next-step plan for lifelong professional growth through MLOps practices.

# Contents


**1 Project Context and Scope** **3**

1.1 Stack Overview . . . 3

**2 Technical Decision and Justification** **3**

2.1 Chosen Technique: SHAP for Local Explanations . . . 3

2.2 Model Choice and Ensemble Rationale . . . 4

2.3 Scalability and Memory Considerations . . . 4

2.4 Comparison with Alternatives . . . 5

2.5 Technical Defense Summary . . . 5

**3 Self-Directed Learning Experience** **5**

3.1 Key Challenges . . . 5

3.2 Learning Resources and Critical Evaluation . . . 6

3.3 Self-Assessment of Learning Outcomes . . . 6

**4 Project Blocks and Structure** **7**

4.1 Repository-Level Blocks . . . 7

4.2 High-Level Architecture Diagram . . . 7

4.3 Module Interaction Diagram . . . 8

4.4 Database ER Diagram . . . 8

4.5 Explainability Screen . . . 9

4.6 Product Tour Screens . . . 9

4.7 Supplementary Architecture Diagram . . . 11

4.8 Prediction Request Sequence . . . 11

4.9 ML Training and Registry Flow . . . 11

4.10 Deployment Boundary Diagram . . . 12

4.11 UML-Style Domain Diagram . . . 12

**5 Reflection and Future Learning Plan** **12**

5.1 Impact on Learning Mindset . . . 12

5.2 Next Learning Goal: MLflow for MLOps . . . 12

5.3 Concrete Plan (4 Weeks) . . . 13

**6 Conclusion** **13**

# 1 Project Context and Scope

EduPredict is a web application that predicts whether a student is *At-Risk* or *Not-At-Risk* based on academic indicators such as attendance, assignments, quizzes, exams, and GPA. The system supports role-based use cases: administrators manage users, teachers submit and review records, and students view their own outcomes. The system consists of:

- A Next.js frontend with role-based dashboards.
- A FastAPI backend providing authentication, academic data management, and ML inference.
- A database layer (SQLite for local development, PostgreSQL for production).

The project required the integration of modern web engineering practices with machine learning inference and explainability. The most significant new technique I independently learned and applied was SHAP-based explainability for tree models, because the problem domain requires transparent decisions and actionable feedback grounded in interpretable evidence.

## 1.1 Stack Overview

The project integrates multiple tools and techniques:

- **Frontend:** Next.js, TypeScript, Tailwind CSS, and charting for dashboards.
- **Backend:** FastAPI, SQLAlchemy, Alembic migrations, and JWT-based RBAC.
- **ML:** scikit-learn for baseline modeling, LightGBM for non-linear modeling, SHAP for explanation, Joblib for artifact storage.
- **Deployment:** Docker for backend packaging and PostgreSQL via Docker Compose.

# 2 Technical Decision and Justification

This section provides a deep technical defense of the chosen technique, demonstrating its fit with system constraints, code structure, and operational requirements.

## 2.1 Chosen Technique: SHAP for Local Explanations

I integrated SHAP (SHapley Additive exPlanations) to provide local, per-student explanations. In EduPredict, prediction accuracy alone is insufficient: teachers must understand *why* a student is flagged at risk to plan interventions. SHAP supports this by

decomposing a prediction into feature contributions. This is a stronger fit than global feature importance because educators need individualized explanations to choose actions like tutoring, counseling, or academic follow-up.

For a model output $f(x)$, SHAP represents the prediction as:

$$f(x) = \phi_0 + \sum_{i=1}^{n} \phi_i, \tag{1}$$

where $\phi_0$ is the expected output and $\phi_i$ is the contribution of feature $i$ for the specific input $x$. This formulation aligns with transparency requirements in educational contexts.

## 2.2 Model Choice and Ensemble Rationale

The inference pipeline uses a simple ensemble of Logistic Regression and LightGBM. The final probability is the mean of the two model probabilities:

$$p = \frac{1}{2} p_{\text{LR}} + \frac{1}{2} p_{\text{LGBM}}. \tag{2}$$

The predicted label is computed with a threshold:

$$\hat{y} = \mathbb{1}[p \geq 0.5]. \tag{3}$$

Logistic Regression provides a stable baseline and interpretable global trends, while LightGBM captures non-linear interactions common in academic performance data (e.g., high GPA with low attendance). SHAP TreeExplainer is optimized for tree models, making LightGBM the most practical choice for efficient, consistent explanations. From a code-structure perspective, the backend isolates ML logic in a dedicated ML module, so the ensemble and explanation logic are encapsulated away from API routing and data access layers, which keeps concerns separated and testing feasible.

## 2.3 Scalability and Memory Considerations

The explanation path must be efficient because it runs alongside API requests. TreeExplainer for LightGBM executes in $O(T \cdot L)$ where $T$ is the number of trees and $L$ is the maximum depth, which is significantly more efficient than kernel-based explainers that scale with the number of samples and features. This makes SHAP feasible for real-time use. In memory terms, the LightGBM model is compact relative to neural alternatives and can be loaded once and cached in the inference process.

## 2.4 Comparison with Alternatives

I considered two alternatives:

- **Pure Logistic Regression with coefficient explanations.** This option is globally interpretable but underfit the data; it missed non-linear patterns and produced weaker predictive performance.
- **LIME or permutation feature importance.** LIME provides local explanations but is stochastic and slower for real-time APIs. Permutation importance is global and does not explain individual predictions. In contrast, SHAP with TreeExplainer provides consistent local explanations with lower latency.

The SHAP + LightGBM choice was more effective because it balanced predictive accuracy with stakeholder-facing transparency, and it integrated efficiently into the API. It also supported a clean code separation: the model registry stores artifacts, inference loads the latest model once, and the API layer returns both prediction and explanation.

## 2.5 Technical Defense Summary

The chosen technique satisfied the following constraints simultaneously:

- **Interpretability:** local explanations for each student using SHAP.
- **Performance:** efficient tree-based explanation with predictable latency.
- **Maintainability:** isolated ML module, registry-based versioning, and minimal coupling with API routing.
- **Scalability:** compatible with CPU-only environments and lightweight container deployment.

# 3 Self-Directed Learning Experience

## 3.1 Key Challenges

1. **Correct interpretation of SHAP values.** I initially treated SHAP as simple feature importance. I had to learn the distinction between global importance and local contribution and how the baseline affects interpretation.
2. **Maintaining feature consistency.** Explanations are only valid if the inference feature order and preprocessing match the training pipeline exactly. I enforced a stable feature list and consistent preprocessing steps in both training and inference paths.

3. **Binary classification output formats.** TreeExplainer can return different shapes for binary tasks. I had to identify the correct class-specific SHAP array to avoid incorrect explanations.

4. **Asynchronous backend integration.** The backend uses async database sessions and dependency injection. I had to ensure inference and explanation logic did not block the event loop and that model loading was cached properly.

5. **Auth and RBAC design.** JWT-based RBAC required careful role checks in API routes so that students only access their own records while admins have broader permissions.

## 3.2 Learning Resources and Critical Evaluation

| Resource Type | Strengths | Limitations |
|---|---|---|
| Official documentation (SHAP, LightGBM, scikit-learn) | Precise definitions, official examples, and correct output formats for explainers. | Dense and time-consuming; assumes prior familiarity with tree-based learning. |
| Tutorials and blog posts | Quick start with runnable code and intuition. | Often omit edge cases, output-shape details, or correct interpretation boundaries. |
| Community forums and GitHub issues | Practical fixes for errors and version-specific quirks. | Inconsistent quality and sometimes outdated or contradictory. |

Overall, official documentation was the most effective for correctness, while community posts were helpful for specific troubleshooting and error resolution.

## 3.3 Self-Assessment of Learning Outcomes

This experience improved my ability to (1) test assumptions with small experiments, (2) map tool choice to system constraints, and (3) separate concerns in code so ML logic, API routing, and persistence evolve independently. I also gained confidence in reading technical documentation critically rather than using it as a checklist.

# 4 Project Blocks and Structure

This section summarizes the project blocks derived from the repository structure and adds additional diagrams to clarify data flow, request lifecycle, and deployment boundaries.

## 4.1 Repository-Level Blocks

| Block | Responsibility |
|---|---|
| Frontend (apps/frontend) | Next.js UI, dashboards, role-based views, charts |
| Backend (apps/backend) | FastAPI API, authentication, ML inference, RBAC |
| Database | SQLite for local dev, PostgreSQL via Docker |
| ML Registry | Stored model artifacts and metadata |

## 4.2 High-Level Architecture Diagram

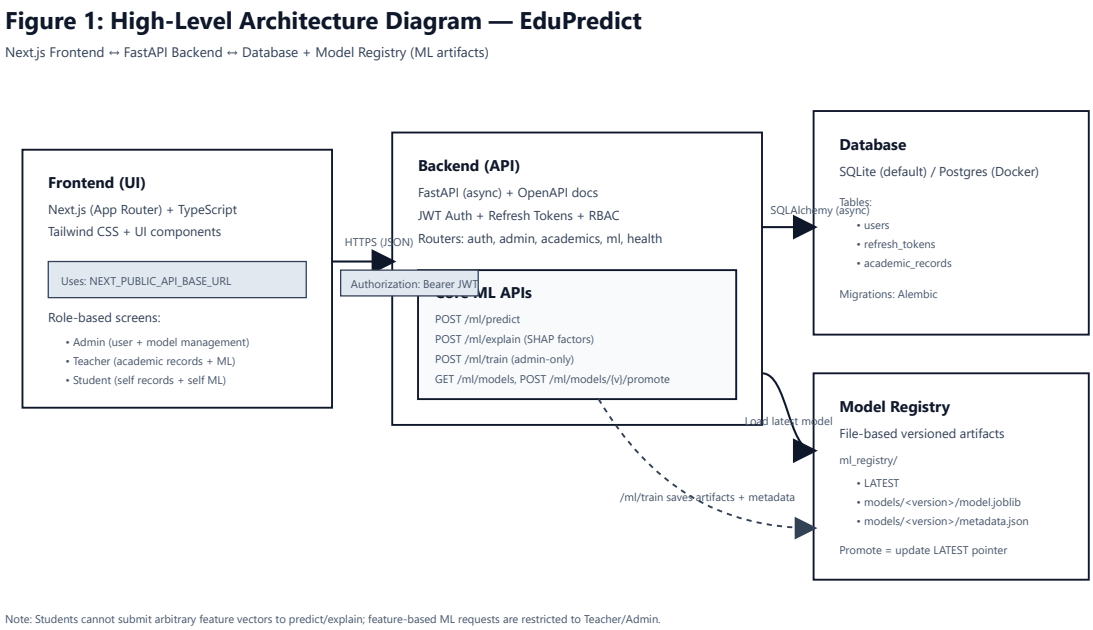



Figure 1: High-level system architecture (from project assets).

## 4.3 Module Interaction Diagram

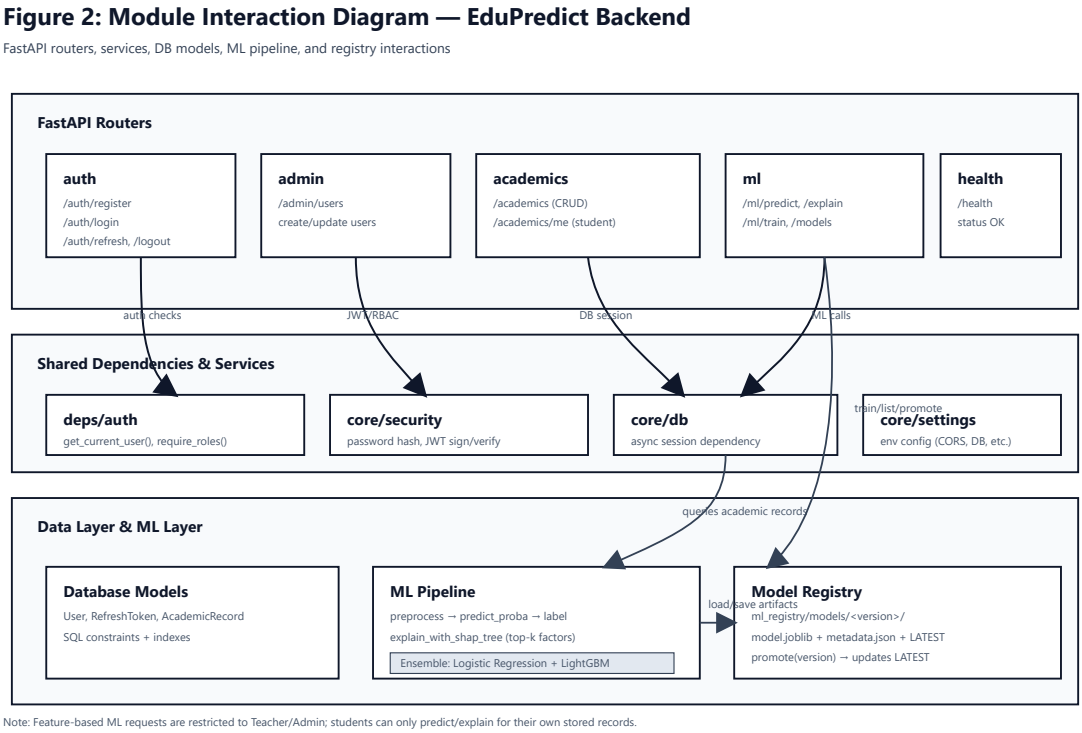



Figure 2: Module interaction overview (from project assets).

## 4.4 Database ER Diagram

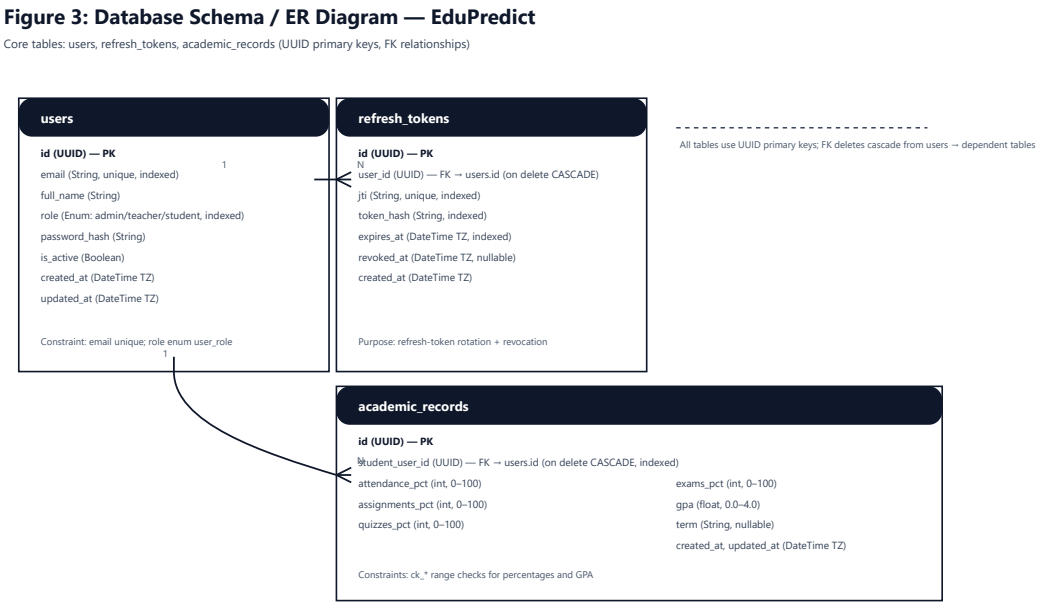



Figure 3: Database ER diagram (from project assets).

## 4.5 Explainability Screen

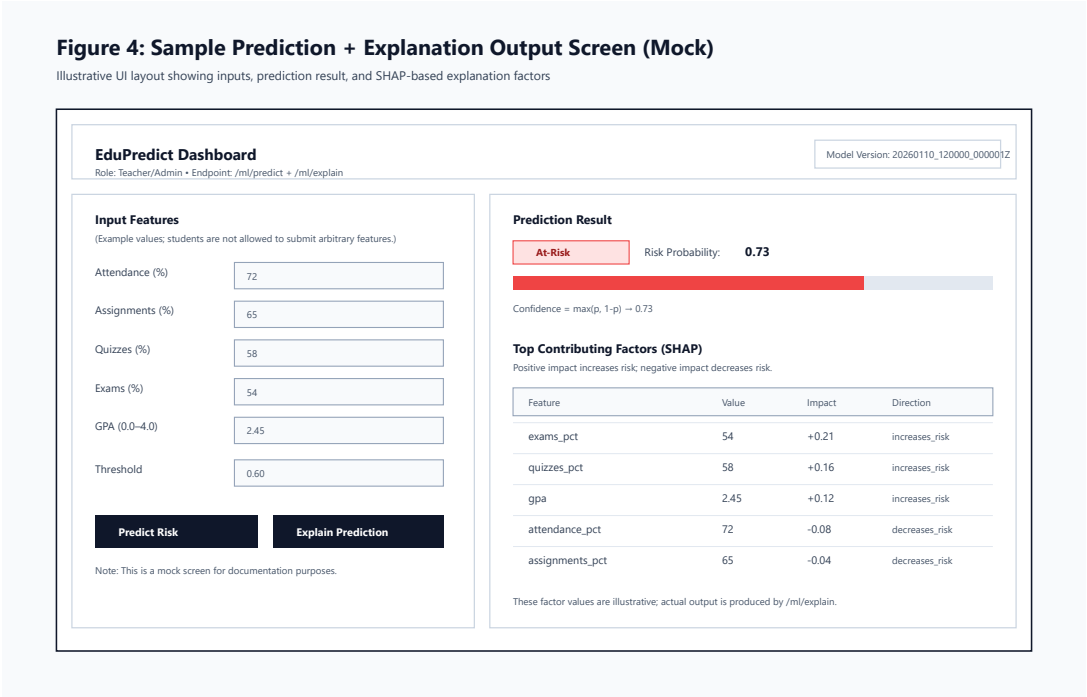



Figure 4: Sample explanation UI (from project assets).

## 4.6 Product Tour Screens

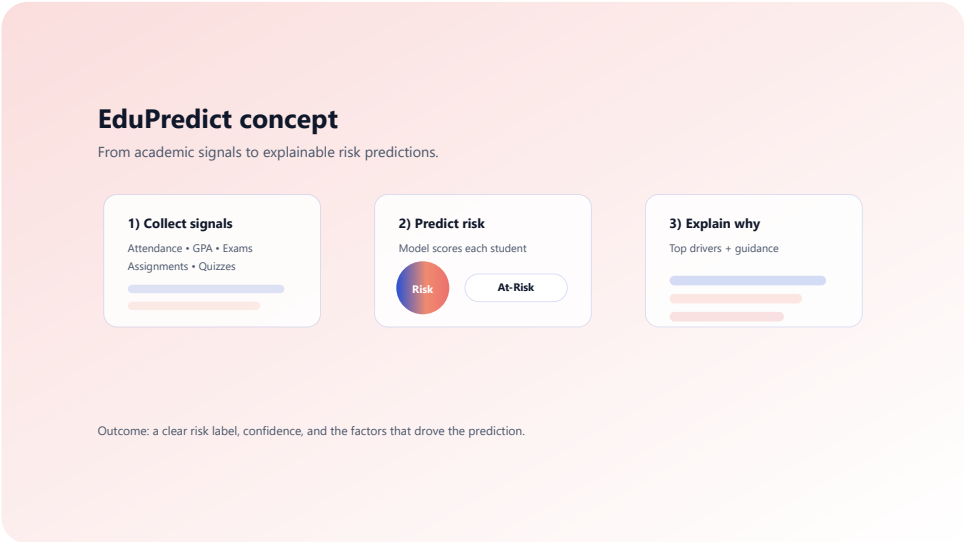
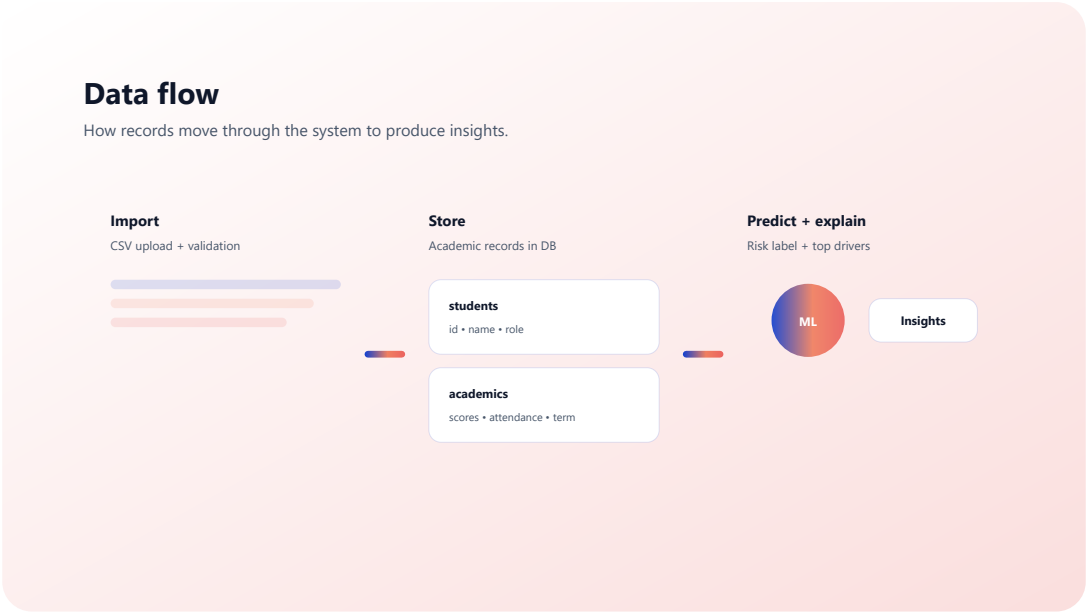
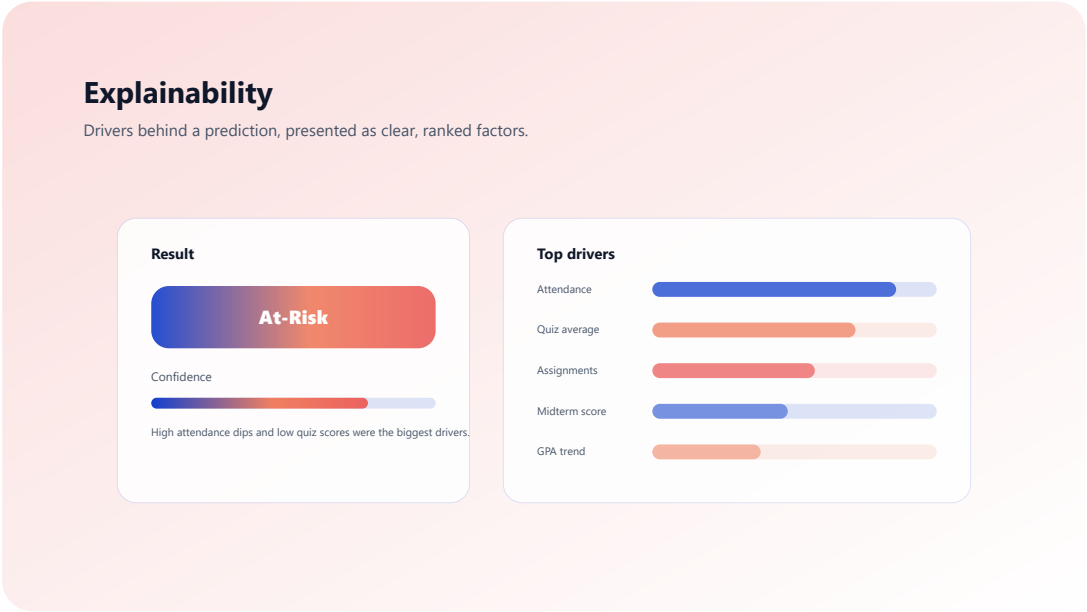



Figure 5: Product concept overview (from project assets).



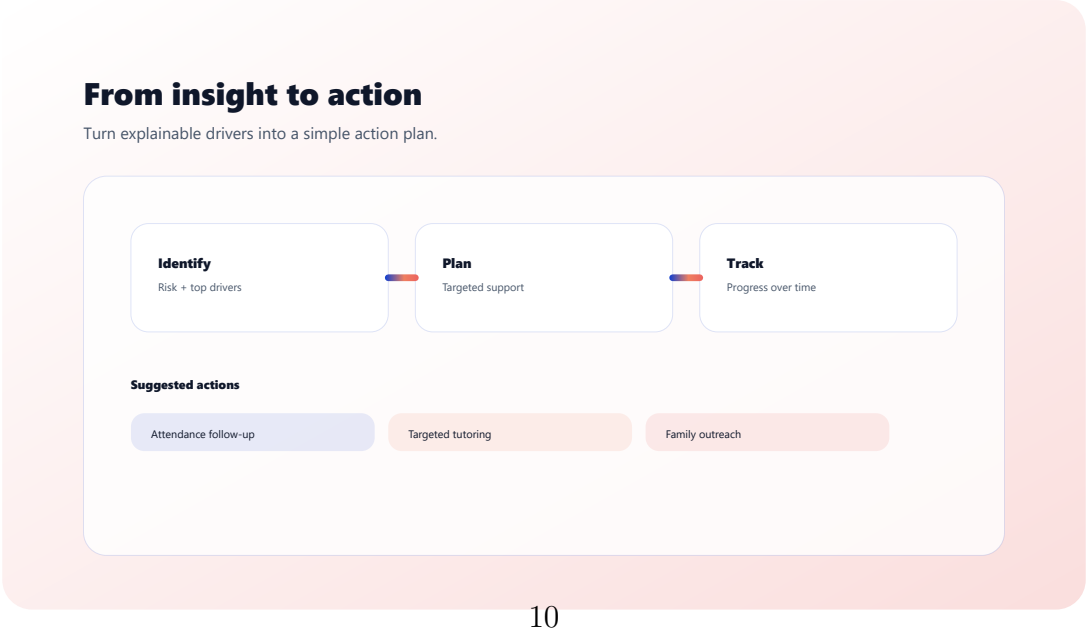






Figure 6: Product tour screens: Data Flow, Explainability, and Intervention Guidance.

## 4.7 Supplementary Architecture Diagram

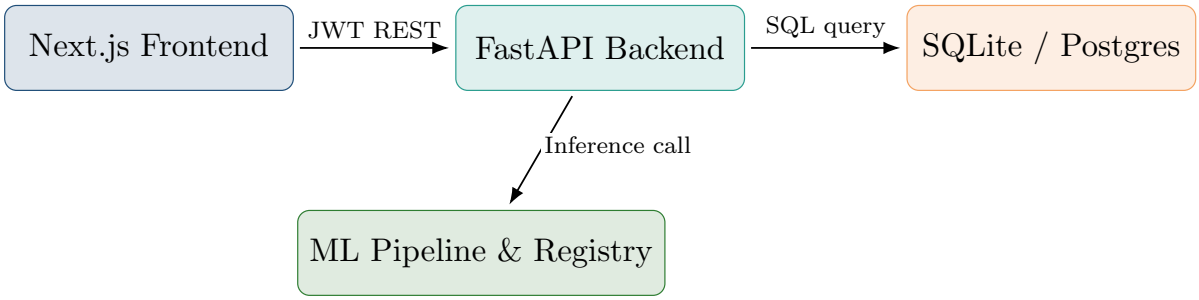



Figure 7: Supplementary architecture block diagram.

## 4.8 Prediction Request Sequence

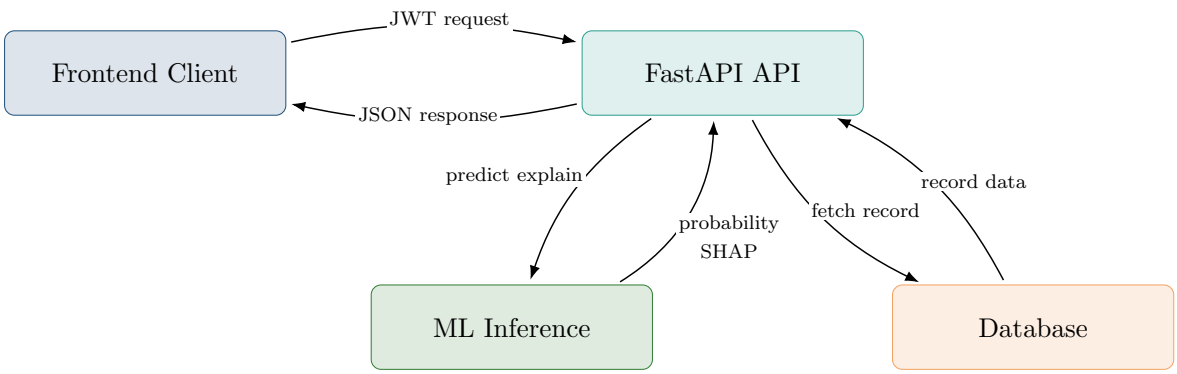



Figure 8: Sequence of a prediction request with explanation payload.

## 4.9 ML Training and Registry Flow

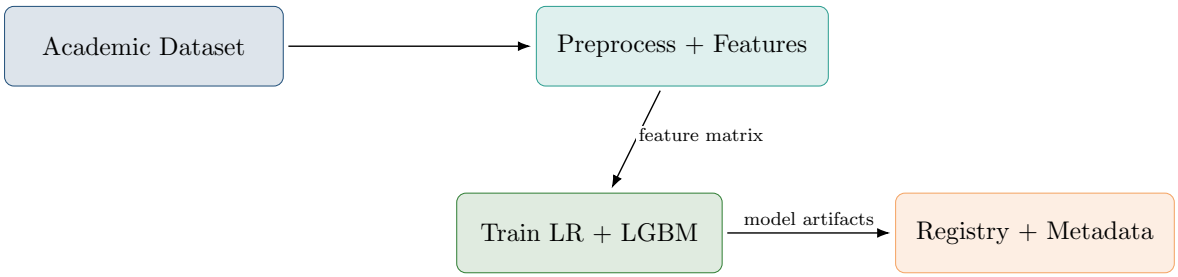



Figure 9: Training and model registry flow used in EduPredict.

## 4.10 Deployment Boundary Diagram

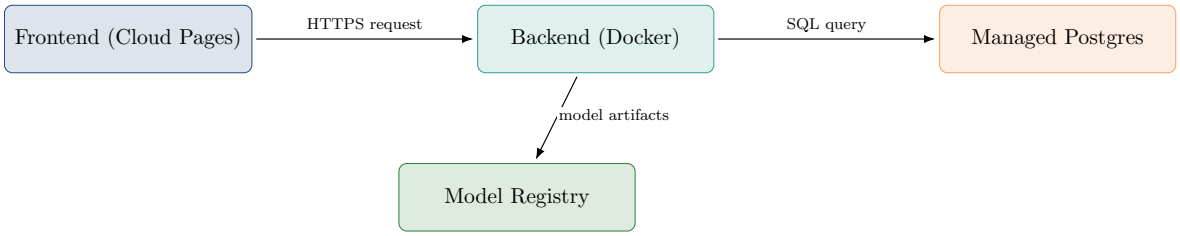



Figure 10: Deployment boundaries and data flows across services.

## 4.11 UML-Style Domain Diagram

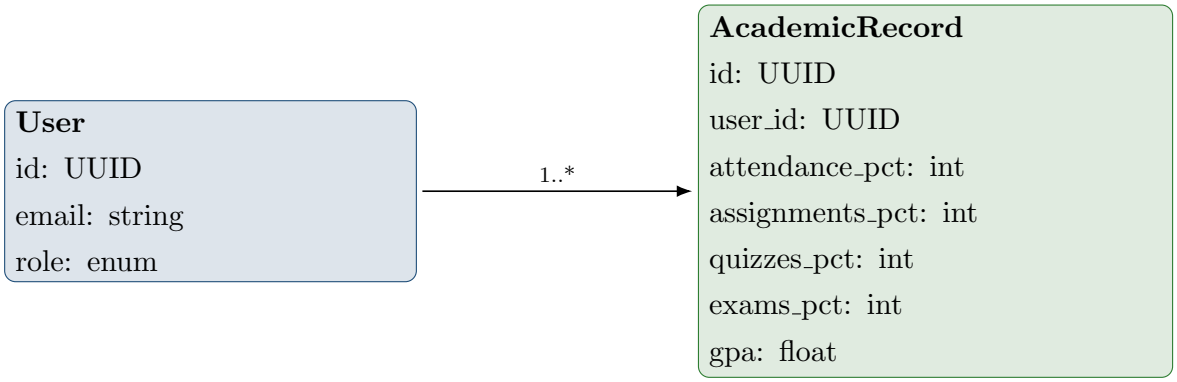



Figure 11: UML-style domain model extracted from backend entities.

# 5 Reflection and Future Learning Plan

## 5.1 Impact on Learning Mindset

This project shifted my learning approach from *tutorial-first* to *requirements-first*. I now begin by clarifying system constraints (accuracy, interpretability, latency, and operational simplicity) and then evaluate tools against those constraints. I also learned to validate assumptions with controlled experiments before committing to a design, which reduced integration risk and improved confidence.

## 5.2 Next Learning Goal: MLflow for MLOps

I plan to learn MLflow to improve experiment tracking, model versioning, and governance. This is a natural next step because EduPredict already maintains a local model registry, but lacks structured experiment lineage and promotion workflows.

### 5.3 Concrete Plan (4 Weeks)

1. **Week 1:** Study MLflow tracking and registry documentation; run a local MLflow server.
2. **Week 2:** Log parameters, metrics, and artifacts during model training.
3. **Week 3:** Register multiple model versions and implement a promotion workflow.
4. **Week 4:** Compare MLflow with the current registry for reproducibility and collaboration; document findings.

The plan applies lessons learned from EduPredict: prioritize documentation, prototype first, and validate each integration incrementally.

## 6 Conclusion

EduPredict required me to learn and integrate explainable AI, modern web frameworks, and API-driven ML deployment. The decision to use SHAP with LightGBM was grounded in the need for transparent, actionable predictions. By critically evaluating learning resources and confronting implementation challenges, I developed a more mature and systematic approach to self-directed learning. This experience prepares me for lifelong professional growth as a software engineer.